

# Repaso para la revisión: Flutter/Dart, comandos, y los cambios más importantes

No es un tutorial de "cómo hacer tal cosa" — es un repaso de qué es cada pieza y por qué está, para poder explicarlo si preguntan. Generado 2026-09-01.

## 1. Flutter y Dart, explicado simple

**Flutter** es un framework de Google para escribir una app con un solo código que compila a Android, iOS, web y escritorio. CowControl apunta a Android + iOS (iOS todavía pendiente, hace falta una Mac con Xcode).

**Dart** es el lenguaje de programación con el que se escribe el código de Flutter (también de Google). Es tipado — cada variable declara qué tipo de dato guarda ( `String` , `int` , `bool` , etc.) — y orientado a objetos, organizado en clases.

**Widget:** en Flutter, todo lo que se ve en pantalla es un "widget" — un botón, un texto, una lista, hasta una pantalla entera. Se arman anidando widgets unos dentro de otros, como cajas dentro de cajas.

**StatelessWidget vs. StatefulWidget:** un *Stateless* no cambia solo — una vez dibujado, queda así hasta que el widget padre lo reconstruye. Un *Stateful* sí puede cambiar con el tiempo (ej: la pantalla de conteo, que se actualiza cada vez que llega una lectura nueva) — internamente llama a `setState()` para avisarle a Flutter "algo cambió, volveme a dibujar".

`pubspec.yaml` : el archivo que lista qué paquetes (librerías) usa el proyecto, con su versión — el equivalente en Flutter a un `package.json` (JavaScript) o `requirements.txt` (Python). Los paquetes vienen de **pub.dev**, el repositorio oficial de Dart/Flutter (como npm para JavaScript) — no hay que "instalar" nada a mano, alcanza con escribir el nombre y la versión en `pubspec.yaml` y correr `flutter pub get` .

| Paquete                             | Para qué se usa                                                                  |
|-------------------------------------|----------------------------------------------------------------------------------|
| <code>supabase_flutter</code>       | Cliente oficial de Supabase — login, base de datos, tiempo real, todo en uno.    |
| <code>flutter_dotenv</code>         | Lee el archivo <code>.env</code> con las credenciales (URL y clave de Supabase). |
| <code>flutter_launcher_icons</code> | Genera el ícono de la app (todos los tamaños) a partir de un solo PNG.           |
| <code>flutter_lints</code>          | Reglas de estilo/errores comunes que usa <code>flutter analyze</code> .          |

`async` / `await` y `Future` : pedirle datos a Supabase tarda (viaja por internet), así que esas operaciones son "asíncronas" — el código no se congela esperando. `Future<Animal>` significa "en algún momento va a haber un Animal disponible, todavía no". `await` le dice a Dart "esperá acá a que esto termine antes de seguir", sin trabar el resto de la app mientras tanto.

## 2. Comandos de terminal más importantes

```
flutter pub get
```

Descarga/instala los paquetes listados en `pubspec.yaml` . Se corre después de agregar un paquete nuevo o de bajar el proyecto en una PC nueva.

```
flutter analyze
```

Revisa todo el código en busca de errores y problemas de estilo, sin ejecutar la app. Es el primer chequeo después de cualquier cambio — tiene que dar **"No issues found!"**.

```
flutter run -d <dispositivo>
```

Compila y corre la app en un dispositivo conectado (celular por USB) o un emulador, en modo "debug". Con **hot reload**: al apretar **r** en la terminal, aplica los cambios de código al instante sin perder el estado de la app en pantalla. **R** (mayúscula) es **hot restart**: reinicia la app de cero, pero sin tener que recompilar todo desde el principio.

```
flutter emulators --launch Pixel_CowControl
```

Prende el emulador de Android que se usa para probar la app sin un celular físico.

```
flutter devices
```

Lista los dispositivos/emuladores conectados en ese momento.

```
flutter build apk --release
```

Genera el instalador final de Android ( **.apk** ) en modo release — optimizado, es el que se comparte con el equipo o se sube a un celular real. Queda en **build\app\outputs\flutter-apk\app-release.apk**.

```
flutter doctor
```

Diagnóstico del entorno de desarrollo — chequea si el SDK de Android, las licencias, etc. están bien instalados.

```
adb install -r app-release.apk
```

Instala un APK directo en un celular conectado por USB (con Depuración USB activada), sin pasar por WhatsApp/Drive. Útil cuando la conexión de **flutter run** es inestable con algún celular en particular.

## Git (control de versiones)

| Comando                              | Para qué                                                                             |
|--------------------------------------|--------------------------------------------------------------------------------------|
| <code>git init</code>                | Empieza a llevar el historial de cambios de una carpeta.                             |
| <code>git add &lt;archivo&gt;</code> | Marca un archivo para incluir en el próximo commit.                                  |
| <code>git commit -m "..."</code>     | Guarda una "foto" del proyecto en ese momento, con un mensaje explicando qué cambió. |
| <code>git push</code>                | Sube los commits nuevos al repositorio de GitHub.                                    |
| <code>git clone &lt;url&gt;</code>   | Descarga una copia completa del proyecto (con todo su historial) desde GitHub.       |

### 3. Estructura del código (carpeta lib/ )

```
lib/  
├─ main.dart           → arranca la app, inicializa Supabase  
├─ theme/              → paleta de colores de toda la app  
├─ models/             → Animal, Campo, Lectura, Vacuna  
├─ services/           → hablan con Supabase (una clase por tema)  
├─ screens/            → cada pantalla  
└─ widgets/           → piezas reutilizables (tarjeta de animal, ícono)
```

- **models/** — clases que representan los datos que vienen de Supabase. Cada una tiene un método `fromMap()` que convierte la respuesta cruda (un `Map`, básicamente un JSON) en un objeto Dart con el que el resto del código trabaja cómodo.
- **services/** — la capa que efectivamente habla con Supabase ( `AuthService`, `CamposService`, `AnimalesService`, `LecturasService`, `VacunasService`, `ConteosService` ). Las pantallas nunca hablan directo con Supabase, siempre pasan por un service — si el día de mañana cambia cómo se pide un dato, solo hay que tocar un lugar.
- **screens/** — cada pantalla: login, selector de campo, planilla general, conteo en vivo, ficha de animal, historial, fertilidad, vacunas, observaciones.
- **widgets/** — piezas chicas reutilizadas entre pantallas.

### 4. Los cambios más importantes hechos en el proyecto

- **Multi-tenant con Auth + RLS:** cada productor tiene su cuenta y solo ve sus propios campos/animales (detalle en la sección 5).
- **fertilidad reemplazó a "condición corporal":** decisión de producto — `fertilidad` sí está en el mockup original.
- **Pantalla 4 (ficha del animal) rediseñada** en 3 botones (Fertilidad/Vacunas/Observaciones), cada uno con su propia pantalla — antes estaba todo junto en una sola.
- **Íconos de categoría propios** (vaca/toro/ternero/ternera/vaquillona), generados con IA, en vez de los símbolos genéricos de Flutter.
- **Recuperación de contraseña:** un link de un solo uso al mail que abre una página web (no la app) para poner la contraseña nueva — mismo estándar que usan bancos y apps grandes (nada de "pregunta secreta", eso está obsoleto desde 2017 según NIST).
- **El bug del permiso INTERNET** (buena para contar si preguntan sobre debugging): mes y medio sin poder registrarse/loguearse desde el APK compartido, pero sí andaba con `flutter run`. Causa: el manifest de Android de *release* nunca declaró el permiso de internet — solo lo tenía el manifest de *debug*, que Flutter agrega solo y por eso nunca se notó probando en la PC.
- **Se sacó id\_animal de datos\_lectura** : nunca lo llenaba nadie — el emparejamiento real entre una lectura y un animal es por `rfid_uid` (el código del tag), no por ese campo.
- **La tabla conteos ahora la escribe la app:** al iniciar/pausar/reanudar/finalizar un conteo desde la Pantalla 3, queda una fila en Supabase (estado `en_curso` / `pausado` / `finalizado` ) que la Raspberry Pi puede consultar para saber a qué conteo mandarle las lecturas que va leyendo.

## 5. Supabase y SQL — lo esencial

**Qué es Supabase:** una base de datos Postgres gestionada, con autenticación, API REST automática, tiempo real (websockets) y almacenamiento de archivos, todo en un solo servicio — hace de "servidor" del proyecto sin tener que programar un backend propio desde cero.

**RLS (Row Level Security):** reglas que Postgres aplica automáticamente para decidir qué **filas** puede ver o tocar cada usuario logueado. En CowControl, cada política dice básicamente "solo si el campo de esta fila pertenece al productor logueado ahora mismo".

```
CREATE POLICY "solo animales de su campo" ON datos_animales
FOR ALL TO authenticated
USING (EXISTS (
  SELECT 1 FROM datos_campo
  WHERE datos_campo.id = datos_animales.campo_id
  AND datos_campo.productor_id = auth.uid()
));
```

En criollo: "dejá hacer cualquier operación sobre una fila de `datos_animales` solo si existe un campo, dueño de esa fila, cuyo `productor_id` sea el mismo que el usuario logueado ahora mismo".

**GRANT:** una capa distinta y más básica que RLS — decide si el rol `authenticated` puede siquiera *intentar* leer/escribir esa tabla, antes de que RLS entre a evaluar nada. Bug real que pasó: faltaba el GRANT en las 4 tablas originales (nadie podía leer nada, error 42501 "permission denied") aunque las políticas RLS estaban perfectas — son dos capas separadas, hacen falta las dos.

**service\_role key vs. anon key :** la Raspberry Pi usa la `service_role key`, que se salta RLS y GRANT a propósito (no puede "loguearse" como un usuario). La app usa la `anon key`, que sí respeta RLS — la `anon key` no es secreta, está pensada para ir en código público; lo que protege los datos de verdad es RLS del lado del servidor.

### Migraciones SQL más importantes que se corrieron

| Qué                             | Resumen                                                                                                                  |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Multi-tenant                    | <code>productor_id</code> en <code>datos_campo</code> + políticas RLS en las 4 tablas originales.                        |
| Condición corporal → fertilidad | <code>ALTER TABLE</code> para sacar una columna y agregar otra con <code>CHECK</code> de valores fijos.                  |
| GRANT explícito                 | <code>GRANT SELECT, INSERT, UPDATE, DELETE</code> en las 4 tablas para el rol <code>authenticated</code> .               |
| Tabla <code>conteos</code>      | <code>campo_id</code> , GRANT, y política RLS agregados — la tabla la había creado un compañero, faltaban esas 3 piezas. |
| Baja de <code>id_animal</code>  | <code>ALTER TABLE datos_lectura DROP COLUMN id_animal</code> — columna sin usar.                                         |

## 6. Preguntas que te pueden hacer (respuestas cortas)

### ¿Por qué Flutter?

Un solo código compila a Android y iOS (las dos plataformas que pide el proyecto), sin mantener dos apps separadas escritas en lenguajes distintos.

### ¿Por qué Supabase y no un servidor propio en la Raspberry Pi?

Supabase ya resuelve base de datos, autenticación, API y tiempo real de forma gestionada y gratuita para este tamaño de proyecto. Armar un servidor propio en la Raspberry Pi habría significado programar todo eso desde cero, y además la Pi tiene una tarea crítica (leer los tags) que no conviene arriesgar sumándole más carga.

### ¿Por qué 4 tablas ( `datos_campo` , `datos_animales` , `datos_lectura` , `datos_vacuna` ) y no menos?

Representan 4 cosas distintas que no tiene sentido mezclar: un establecimiento, un animal, una lectura puntual del lector, y el historial de vacunas de un animal (que puede tener varias en el tiempo). Separarlas es la forma estándar de organizar una base de datos relacional — evita repetir datos y permite que un animal tenga cualquier cantidad de vacunas sin duplicar el resto de sus datos en cada fila.

### ¿Qué pasa si una Raspberry Pi mal configurada escribe en el campo de otro productor?

Hoy no hay nada de software que lo impida — la `service_role` key `bypasses` RLS a propósito para que la Pi pueda insertar sin loguearse. Es un límite real, aceptable para un prototipo de secundaria sin actores externos hostiles. En un producto real, cada Raspberry física tendría su propia credencial, atada del lado del servidor a un único `campo_id` permitido.

### ¿Cómo instalás o actualizás un paquete de Flutter?

Se agrega el nombre y la versión en `pubspec.yaml` , dentro de `dependencies:` , y se corre `flutter pub get` — Flutter lo descarga solo desde `pub.dev`. No hace falta instalar nada a mano ni buscar un instalador.

### ¿Y para iOS?

Hace falta sin excepción una Mac con Xcode — es un requisito de Apple, no se puede compilar para iPhone desde Windows. Está identificado y con plan, solo falta el acceso al hardware.

Para el detalle línea por línea de cada archivo, protocolos de comunicación, y el diccionario de campos completo, está `explicacion_cada_funcion.pdf` en esta misma carpeta — este documento es un repaso rápido, no lo reemplaza.